CEUE203 – OOP

PRACTICAL - 8

Roll No.: 25IT037

Aim:

To implement exception handling in Java using custom exceptions, checked exceptions, finally, and try-with-resources with AutoCloseable resources.

Program 1: Guarded Calculator

Code Screenshot:

```
import java.util.*;


class DivideByZeroException extends Exception {
    DivideByZeroException(String message) {
        super(message);
    }
}


public class pra_8GuardedCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        boolean success=false;
        while (!success) {
            try {
                System.out.print("Enter first number: ");
                double a =sc.nextDouble();
                System.out.print("Enter operator (+, -, *, /): ");
                char op =sc.next().charAt(0);
                System.out.print("Enter second number: ");
                double b =sc.nextDouble();
                double result;
                switch (op){
```

```
                    case '+': result =a+b;break;
                    case '-': result =a-b;break;
                    case '*': result =a*b;break;
                    case '/':
                        if (b == 0) throw new DivideByZeroException("Cannot divide by zero.");
                        result =a/b;
                        break;
                    default: throw new IllegalArgumentException("Invalid operator.");
                }
                System.out.println("Result: " + result);
                success =true;
            } catch (InputMismatchException e) {
                System.out.println("Invalid number input.");
                sc.nextLine();
            } catch (DivideByZeroException e) {
                System.out.println(e.getMessage());
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            } finally {
                System.out.println("Attempt logged.");
            }
        }
        sc.close();
    }
}
```

Output Screenshot:

```
Enter first number: 10
Enter operator (+, -, *, /): /
Enter second number: 0
Cannot divide by zero.
Attempt logged.
Enter first number: 10
Enter operator (+, -, *, /): /
Enter second number: 2
```

```
  Result: 5.0
  Attempt logged.
```

Program 2: Stock Issue

Code Screenshot:

```
import java.util.*;

class OutOfStockException extends Exception {

    private int shortfall;


  OutOfStockException(String message,int shortfall) {

        super(message);

        this.shortfall=shortfall;

    } int getShortfall() {

       return shortfall;

    }

}


class InvalidQuantityException extends Exception {

    InvalidQuantityException(String message) {

        super(message);

    }

}

class Warehouse {

    private Map<String, Integer> stock = new HashMap<>();

 Warehouse() {

        stock.put("Pen",10);

        stock.put("Book",5);

        stock.put("Bag",2);

    }

 void issue(String item, int qty) throws OutOfStockException, InvalidQuantityException {

        if (qty<=0)

            throw new InvalidQuantityException("Quantity must be greater than zero.");

        int available = stock.getOrDefault(item, 0);
```

```
        if (qty>available)
            throw new OutOfStockException("Not enough stock for " + item + ". Shortfall: " + (qty -
available), qty - available);
        stock.put(item,available - qty);
        System.out.println("Issued " +qty+ " "+item+".");
    }
}


public class pra_8stockissue {
    public static void main(String[] args) {
        Warehouse warehouse = new Warehouse();
        String[][] requests = {
            {"Pen", "4"},
            {"Book", "8"},
            {"Bag", "0"},
            {"Bag", "1"}
        };
      for (String[] request : requests) {
            try {
                warehouse.issue(request[0], Integer.parseInt(request[1]));
            } catch (OutOfStockException e) {
                System.out.println("Out of stock: " + e.getMessage());
            } catch (InvalidQuantityException e) {
                System.out.println("Invalid quantity: " + e.getMessage());
            }
        }
    }
}
```

Output Screenshot:

```
Issued 4 Pen.
Out of stock: Not enough stock for Book. Shortfall: 3
```

```
  Invalid quantity: Quantity must be greater than zero.
  Issued 1 Bag.
```

Program 3: AutoCloseable Resource

Code Screenshot:

```
class MyResource implements AutoCloseable {

    MyResource() {

        System.out.println("Resource opened.");

    }  void use() {

        System.out.println("Using resource.");

        throw new RuntimeException("Error inside try block.");

    }

  public void close() {

       System.out.println("Resource closed.");

    }

}

public class pra_8AutoCloseableResource {

    public static void main(String[] args) {

        try (MyResource resource = new MyResource()) {

            resource.use();

        } catch (RuntimeException e) {

            System.out.println("Original error: "+e.getMessage());

        }

    }

}
```

Output Screenshot:

```
  Resource opened.
  Using resource.
  Resource closed.
  Original error: Error inside try block.
```